

Unity Game Development

Lab Manual

Experiments with Complete Solutions, Elaboration, and C# Programs

Student Name	Md Shohel Hossain
Registration No.	24BCG10042
Course / Section	BL2026270100720 C11 + C12
Subject	Game Programming using Unity
Unity Version	2022.3.9f1 LTS
Reference Project	Football Game – player vs AI opponent
Submission Date	06 October 2026

Table of Contents

Experiment 1 — Asset Store, Textures, Materials, and Shaders	3
1(a) Importing a texture and applying it to a model's material	4
1(b) Creating and modifying materials with Unity's Material Editor	5
1(c) Applying different shaders and observing visual differences	6
Experiment 2 — Terrain, Character Import, Audio, Lighting, and Level Layout	7
Creating terrain and importing a rigged, animated character	8
2(a) Importing audio and attaching it to objects	9
2(b) Importing terrain heightmaps	10
2(c) Experimenting with lighting setups	11
2(d) Creating a basic level layout	12
Experiment 3 — Scripting Object Movement, Physics, Audio Triggers, Scene Transitions, and Character Control	12
3(a) Physics — gravity, push, and pull forces	13
3(b) Audio triggered by movement/interaction events	14
3(c) Moving between scenes/levels by script	15
3(d) Scripting animations based on events/actions	15
3(e) Full character: move left/right, jump, and crouch	16
3(f) Smooth camera following the player	18
Experiment 4 — Platforms and Obstacles with Collision Detection	19
Experiment 5 — Animating the Player Character (Running, Jumping)	21
Experiment 6 — Animation Transitions for Smoother Character Movement	23
Experiment 7 — Player Movement with Unity's Input System (Speed & Acceleration Tuning)	24
Experiment 8 — Measuring CPU Usage Across Game Scenarios	26
Experiment 9 — Monitoring GPU Usage and Analyzing Memory Consumption	28
Overall Conclusion (Experiments 8–9)	29
Appendix — Mapping to the Lab Assessment questions	29

Unity Game Development — Lab Manual

Experiments with Complete Solutions, Elaboration, and C# Programs

This manual provides a worked solution for each experiment: the aim, underlying theory, step-by-step Unity Editor procedure, complete C# code where scripting is involved, and the expected observation/output, in the standard practical-record format. Blue boxes headed **In my football project** show where the experiment is applied in my own Unity project (*rebuild football*, Unity 2022.3.9f1); dashed boxes mark where the Unity screenshot of that experiment is pasted.

Note: Experiments 3(a)–3(f), 5 and 6 use the classic `Input` class: set *Edit* → *Project Settings* → *Player* → *Active Input Handling* to **Both** (or *Input Manager (Old)*). Experiment 7 needs the *Input System* package, so choose **Both** when you use all experiments in one project.

Experiment 1 — Asset Store, Textures, Materials, and Shaders

Aim

To explore Unity's Asset Store, import external assets into a project, and understand how textures, materials, and shaders combine to determine an object's final appearance.

Theory

Unity separates *appearance* into three layered concepts. A **Shader** is a small GPU program that defines *how* light interacts with a surface (its lighting model and available input slots). A **Material** is an *instance* of a shader with specific values plugged into those slots (a color, a texture, a smoothness value). A **Texture** is an image asset that can be plugged into one of a material's slots (most commonly the Albedo/Base Map slot). The **Asset Store** (accessible via *Window* → *Asset Store*, or the standalone store website) is Unity's marketplace for pre-made models, textures, shaders, audio, and tools, importable directly into a project via the **Package Manager** → **My Assets** tab.

Procedure — importing an Asset Store package

1. Open *Window* → *Asset Store* (or visit `assetstore.unity.com` in a browser while signed into the same Unity ID as the Editor).
2. Search for a free asset (e.g., a texture pack or simple prop pack) and click **Add to My Assets**.
3. In the Editor, open *Window* → *Package Manager*, switch the dropdown to **My Assets**, select the package, and click **Download**, then **Import**.
4. In the **Import Unity Package** dialog, review the file list and click **Import** to bring the assets into your *Assets/* folder.

[Insert screenshot]

Asset Store package imported – Package Manager (My Assets) and Project window

1(a) Importing a texture and applying it to a model's material

Procedure

1. Drag an image file (.png/.jpg) into Assets/Textures/. Unity's Texture Importer processes it automatically.
2. Select the imported texture in the Project window; in the Inspector confirm **Texture Type = Default**, and set **Max Size/Compression** appropriately for the target platform.
3. Create a new Material: Assets → Create → Material, name it (e.g., Crate_Mat).
4. With the material selected, drag the imported texture onto the **Albedo** slot in the Inspector.
5. Drag Crate_Mat onto a model/primitive in the Scene (or assign it via the model's MeshRenderer → Materials array in the Inspector).

Verifying and tweaking the assignment from script

```
using UnityEngine;

public class TextureApplier : MonoBehaviour
{
    public Texture2D crateTexture;

    void Start()
    {
        Renderer rend = GetComponent<Renderer>();
        // Assign the texture to the material's main (Albedo) slot at runtime
        rend.material.mainTexture = crateTexture;

        // Adjust tiling/offset so the texture repeats or shifts across the surface
        rend.material.mainTextureScale = new Vector2(2f, 2f); // tile twice in U and V
        rend.material.mainTextureOffset = new Vector2(0.1f, 0f);
    }
}
```

Observation: The cube's surface displays the imported image instead of the default grey/checker material. Changing **Tiling** in the Inspector (or `mainTextureScale` in code) repeats the texture across the surface; changing **Offset** slides it.

In my football project: the pitch markings come from a pitch texture (e.g. `gro_1.jpg`, Fig. 1.1) assigned to the Albedo slot of a material that is applied to the **floor** plane; the two players use the Mixamo diffuse/normal textures through the materials `Ch38_body` and `Ch42_Body`.

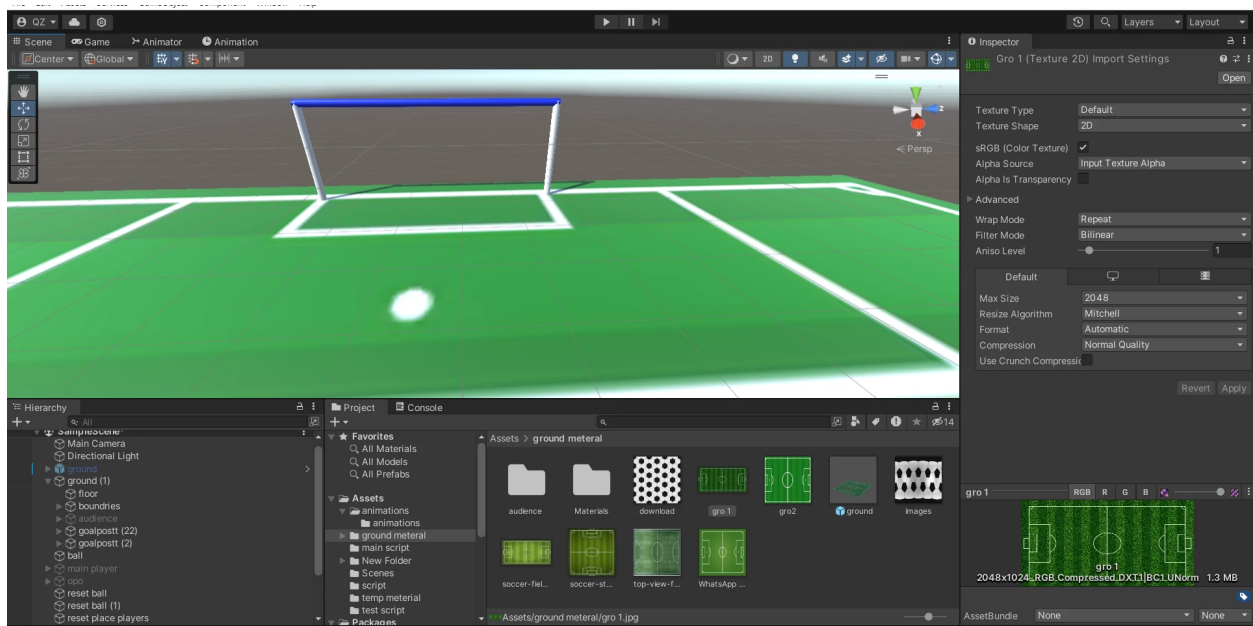


Fig. 1.1 – Texture Importer for the pitch texture gro 1.jpg (Texture Type Default, Wrap Mode Repeat, Max Size 2048, Normal Quality compression); the Project window shows the other pitch textures and the goal post in the Scene view.

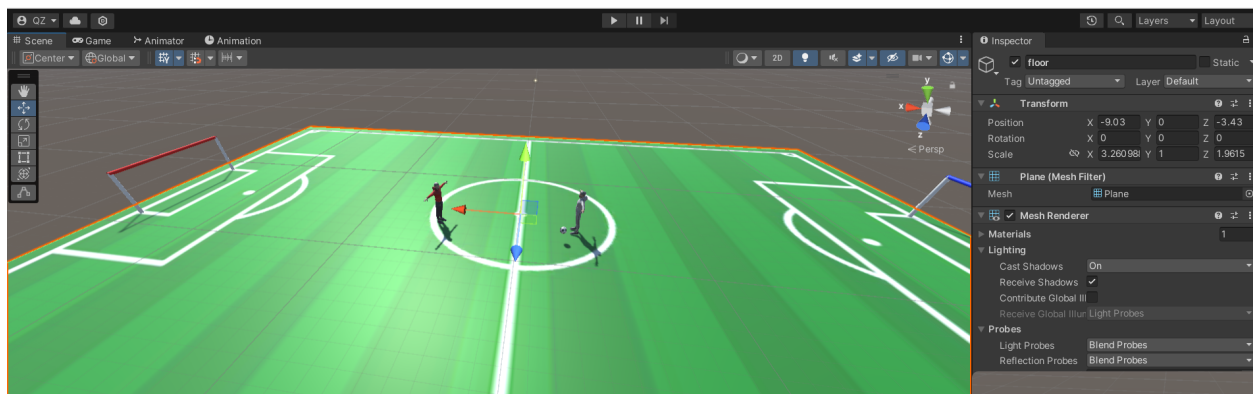


Fig. 1.2 – My project: the **floor** object with its Mesh Renderer, Materials and Lighting (Cast/Receive Shadows) settings in the Inspector.

1(b) Creating and modifying materials with Unity's Material Editor

Procedure

1. Assets → Create → Material, name it Metal_Mat.
2. In the Inspector, set the **Shader** dropdown to Universal Render Pipeline/Lit (or Standard in the Built-in pipeline).
3. Adjust: **Base Map/Albedo** color, **Metallic** slider (0 = non-metal, 1 = fully metallic), **Smoothness** slider (0 = rough/matte, 1 = mirror-like), and optionally plug in a **Normal Map** for surface detail.
4. Drag the material onto an object and orbit the Scene View camera to see how Metallic/Smoothness change the specular highlight.

Modifying material properties by script

```

using UnityEngine;

public class MaterialEditorDemo : MonoBehaviour
{
    public Material metalMaterial;

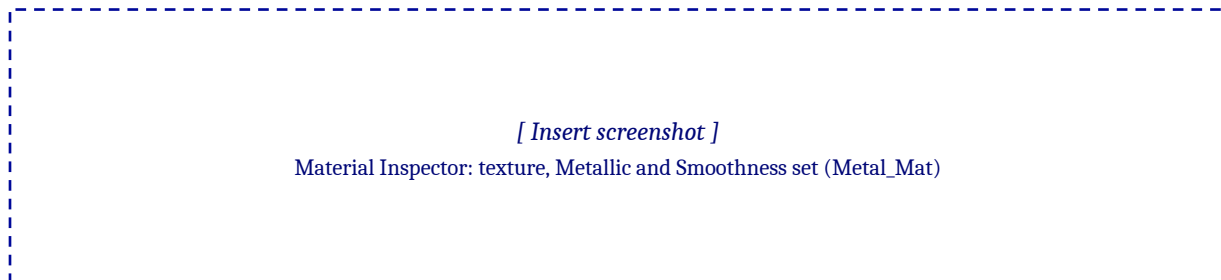
    void Start()
    {
        Renderer rend = GetComponent<Renderer>();
        rend.material = metalMaterial;

        rend.material.color = new Color(0.7f, 0.7f, 0.75f);
        rend.material.SetFloat("_Metallic", 0.9f);
        rend.material.SetFloat("_Smoothness", 0.6f);    // "_Glossiness" in the Built-in Standard shader
    }

    // Interactive experimentation - cycle smoothness with a key press
    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Space))
        {
            Renderer rend = GetComponent<Renderer>();
            float current = rend.material.GetFloat("_Smoothness");
            float next = (current >= 1f) ? 0f : current + 0.25f;
            rend.material.SetFloat("_Smoothness", next);
            Debug.Log("Smoothness now: " + next);
        }
    }
}

```

Observation: Increasing Metallic makes the surface reflect the environment/skybox instead of showing flat diffuse color; increasing Smoothness tightens and brightens the specular highlight, simulating a more polished surface.



1(c) Applying different shaders and observing visual differences

Procedure

1. Create three materials from the same base color/texture, but assign each a different shader: Universal Render Pipeline/Lit (full PBR lighting), Universal Render Pipeline/Unlit (ignores all scene lighting, flat color), and Universal Render Pipeline/Simple Lit (a cheaper, non-PBR lighting model suited to mobile/stylized looks). In the Built-in pipeline use Standard, Unlit/Texture and Legacy Shaders/Diffuse.
2. Apply each material to an identical sphere placed side by side, under the same Directional Light.
3. Compare: the **Lit** sphere shows realistic shading, specular highlight, and shadow response; the **Unlit** sphere appears the same flat brightness everywhere regardless of light angle; the **Simple Lit**

sphere shows basic shading without full PBR reflections.

Script to swap shaders/materials at runtime for side-by-side comparison

```
using UnityEngine;

public class ShaderComparison : MonoBehaviour
{
    public Material[] shaderVariants; // assign Lit, Unlit, SimpleLit materials in Inspector
    private int currentIndex = 0;

    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Tab))
        {
            currentIndex = (currentIndex + 1) % shaderVariants.Length;
            GetComponent<Renderer>().material = shaderVariants[currentIndex];
            Debug.Log("Now showing shader: " + shaderVariants[currentIndex].shader.name);
        }
    }
}
```

Observation / Result: The same geometry and base color can look dramatically different purely based on the shader assigned — confirming that a shader defines the *lighting model*, while the material only supplies *parameter values* into that model.

Shader	Reacts to light	Shadows received	Relative GPU cost	My observation
Standard / Lit	Yes (diffuse + specular)	Yes	Higher	
Simple Lit / Diffuse	Yes (simple)	Yes	Medium	
Unlit	No (flat)	No	Lowest	

[Insert screenshot]

Same object with Lit, Simple Lit and Unlit shaders side by side

Experiment 2 — Terrain, Character Import, Audio, Lighting, and Level Layout

Aim

To create a Unity terrain, import a rigged and animated character model, and populate a basic level with audio, lighting, and props.

Theory

Unity's Terrain GameObject stores a heightmap-based surface in a `TerrainData` asset, which can be sculpted by hand or generated from an imported heightmap image. Rigged, animated characters are imported as `.fbx` files whose **Rig** – **Animation Type** is set to `Humanoid` or `Generic`, after which Unity extracts individual animation clips for use in an Animator Controller. A complete level combines terrain, static props (prefabs), lighting, audio, and one or more characters into a coherent playable space.

Creating terrain and importing a rigged, animated character

Procedure

1. `GameObject` → `3D Object` → `Terrain` to create a new terrain in the scene.
2. Use the Terrain Inspector's **Raise/Lower Terrain** brush to sculpt hills and valleys; use **Paint Texture** to blend a grass/dirt splat map across the surface.
3. Import a rigged character `.fbx` (from the Asset Store, Mixamo, or a supplied sample) into `Assets/Models/Character/`.
4. Select the imported model; in the **Rig** tab set **Animation Type = Humanoid**, click **Apply**, then open **Configure...** to verify Unity's automatic bone mapping (or map manually if auto-detection fails).
5. In the **Animation** tab, confirm the imported clips (e.g., `Idle`, `Walk`, `Run`, `Jump`) are correctly trimmed from the source take; tick **Loop Time** for looping clips.
6. Drag the character into the scene, positioned on the terrain surface.

Observation: After import, expanding the model asset in the Project window reveals child sub-assets: the mesh, the Avatar, and each extracted animation clip — confirming Unity parsed both the skeleton and the baked animation data from the single `.fbx` file.

In my football project: both characters are Mixamo humanoid rigs (bone names `mixamorig7:Hips`, `LeftUpLeg`, `Spine...`). Clips used: *breating idle*, *Strike Foward Jog*, *Dribble*, *kick*, *Soccer Tackle*, *Soccer Spin*, stored in `Assets/animations`.

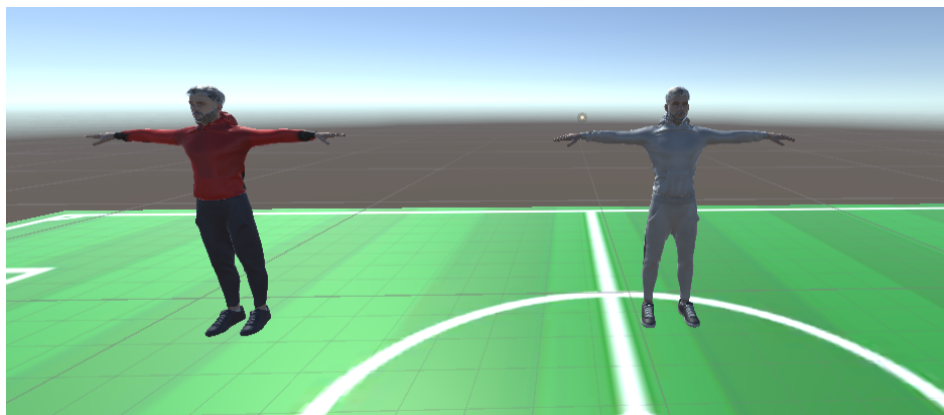


Fig. 2.1 – Imported, rigged Mixamo characters placed on the pitch (opponent in red, player in grey).

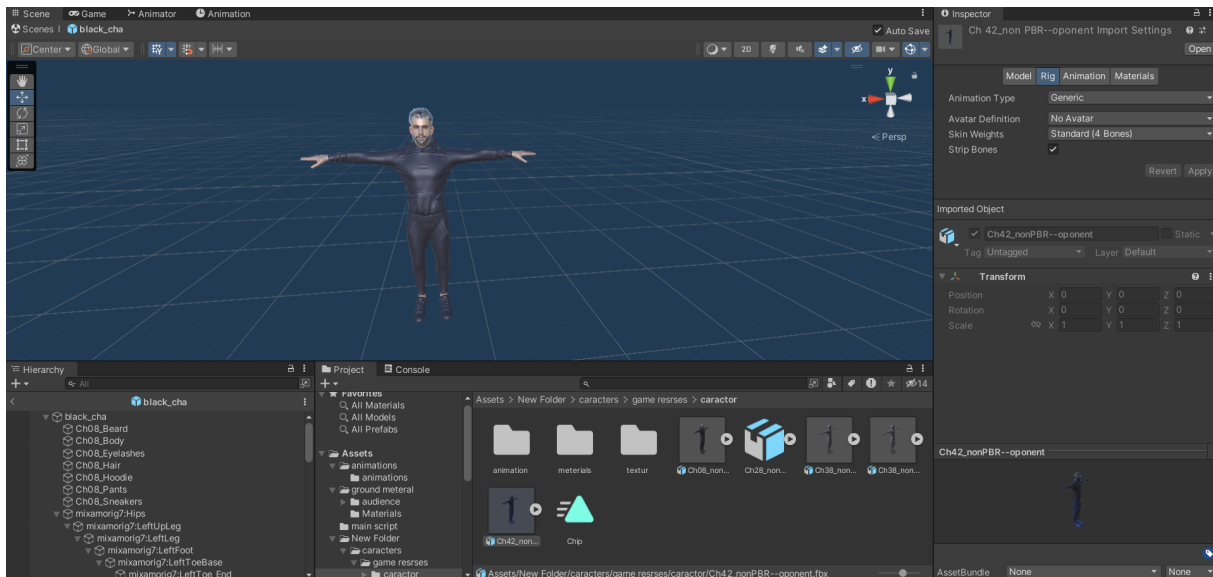


Fig. 2.2 – Model Importer, **Rig** tab of the opponent character FBX (Ch42_nonPBR--oponent); the Hierarchy shows the imported Mixamo bone hierarchy (mixamorig7:Hips → LeftUpLeg → LeftLeg → LeftFoot...).

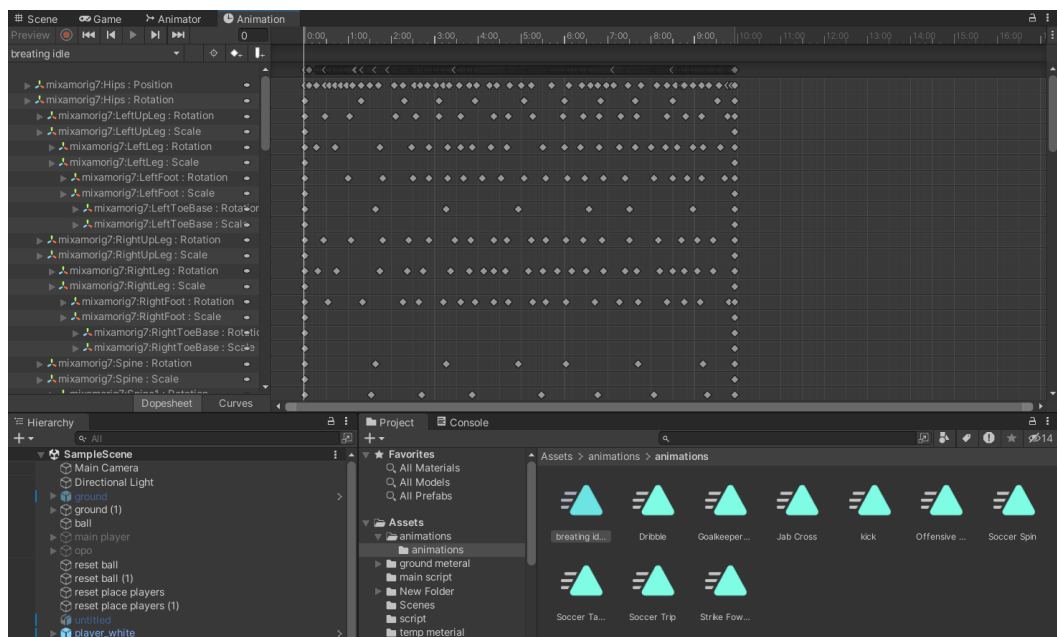


Fig. 2.3 – Animation window showing the bone curves of the breating idle clip and the imported clips in Assets/animations.

2(a) Importing audio and attaching it to objects

Procedure

1. Drag a .wav or .mp3 file into Assets/Audio/. Unity's Audio Importer processes it into an AudioClip.
2. Select the clip; in the Inspector set **Load Type** (Decompress On Load for short SFX, Streaming for long music tracks) and **Force To Mono** if a 3D positional sound doesn't need stereo.

3. Select the target GameObject (e.g., a torch prop or the character), Component → Audio → Audio Source, and assign the clip to the **AudioClip** field.

Script — attaching and triggering a sound effect

```
using UnityEngine;

public class PropSoundEffect : MonoBehaviour
{
    public AudioClip ambientClip;
    private AudioSource source;

    void Start()
    {
        source = gameObject.AddComponent<AudioSource>();
        source.clip = ambientClip;
        source.loop = true;
        source.spatialBlend = 1f;           // 3D positional sound
        source.minDistance = 2f;
        source.maxDistance = 15f;
        source.Play();
    }
}
```

Observation: Walking the Scene View camera (or the player, in Play Mode) closer to and farther from the object causes the sound's volume to attenuate realistically, because `spatialBlend = 1` enables 3D distance-based rolloff.

[Insert screenshot]

Audio clip import settings and Audio Source component on the object

2(b) Importing terrain heightmaps

Procedure

1. Obtain or generate a grayscale heightmap image (brighter pixels = higher elevation), sized as a power-of-two plus one (e.g., 513×513).
2. Select the Terrain GameObject, open the **Terrain Settings** tab, and under **Import Raw** browse to the heightmap file (.raw format is most reliable; 16-bit, matching byte order to the source).
3. Set the terrain's **Width/Length/Height** to match the intended real-world scale, then click **Import**.
4. Adjust **Heightmap Resolution** and **Pixel Error** to trade off sculpting detail against runtime performance.

Observation: The flat terrain reshapes into the mountains/valleys encoded by the heightmap image's brightness values, giving a natural-looking landscape without manual sculpting.

[Insert screenshot]

Import Raw dialog and the terrain after the heightmap is applied

2(c) Experimenting with lighting setups

Procedure

1. Add a **Directional Light** to represent the sun; rotate it and observe how shadow direction and surface shading change across the whole terrain simultaneously.
2. Add a **Point Light** inside/near a prop (e.g., a torch or lamp) with a warm color and moderate range.
3. Add a **Spot Light** aimed at the character to simulate a flashlight or dramatic highlight.
4. Open **Window** → **Rendering** → **Lighting**, switch **Environment Lighting Source** between Skybox, Gradient, and Color, observing how ambient fill light changes in shadowed areas.

Script — cycling lighting setups for comparison

```
using UnityEngine;

public class LightingComparison : MonoBehaviour
{
    public Light sun;
    public Color[] presetColors = { Color.white, new Color(1f, 0.7f, 0.4f), new Color(0.4f, 0.5f, 1f) };
    private int index = 0;

    void Update()
    {
        if (Input.GetKeyDown(KeyCode.L))
        {
            index = (index + 1) % presetColors.Length;
            sun.color = presetColors[index];
            Debug.Log("Sun color set to preset " + index);
        }
    }
}
```

Observation: Warm-colored light casts an orange/amber tone across the terrain and props (evening mood); cool-colored light produces a blue, moonlit feel — demonstrating how lighting alone changes a scene's atmosphere without touching any geometry.

In my football project: the scene uses one Directional Light; the floor has *Cast Shadows = On* and *Receive Shadows* ticked, so the players' shadows fall on the pitch (Fig. 1.2).

[Insert screenshot]

Two lighting setups (day vs. night) with the Light Inspector

2(d) Creating a basic level layout

Procedure

1. Populate the terrain with prop prefabs (rocks, crates, trees) via drag-and-drop, using grid snapping and vertex snapping to arrange them.
2. Place the animated character at a defined **Start** point and mark an **End**/objective point with a trigger volume.
3. Add the lighting and audio elements configured above.
4. Bake or preview lighting (Generate Lighting) and playtest by pressing **Play**.



Fig. 2.4 – Level running in Play mode: pitch, player, opponent and ball at the kick-off position.

Observation / Result: A cohesive small level emerges combining sculpted terrain, an animated character, ambient and positional audio, and directional/point lighting — the minimum viable “vertical slice”.

Experiment 3 — Scripting Object Movement, Physics, Audio Triggers, Scene Transitions, and Character Control

Aim

To write and test a series of C# scripts covering keyboard-driven object movement, physics forces, audio-on-event triggers, scene switching, animation control, full character movement, and smooth camera follow.

Base task — moving a cube forward/backward with the arrow keys

Procedure

1. Create a Cube (GameObject → 3D Object → Cube).
2. Create a C# script `SimpleMover.cs`, attach it to the Cube.

```

using UnityEngine;

public class SimpleMover : MonoBehaviour
{
    public float speed = 5f;

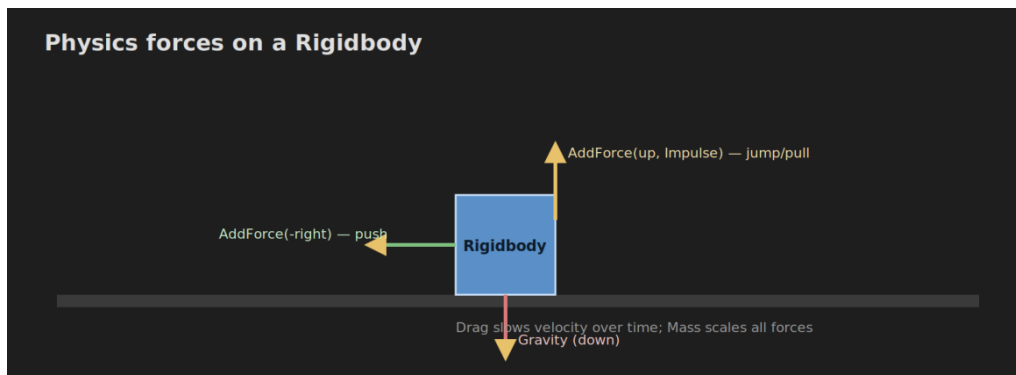
    void Update()
    {
        float moveInput = Input.GetAxis("Vertical"); // Up Arrow = 1, Down Arrow = -1
        Vector3 movement = transform.forward * moveInput * speed * Time.deltaTime;
        transform.Translate(movement, Space.World);
    }
}

```

Observation: Pressing the Up Arrow moves the cube forward along its local forward axis; Down Arrow reverses it. Because `Time.deltaTime` scales the movement, speed stays consistent across different frame rates.

In my football project: the player is moved in `control.cs/movement.cs` with the same `Input.GetAxis("Horizontal"/"Vertical")` pattern, and rotated smoothly towards the movement direction with `Quaternion.RotateTowards`.

3(a) Physics — gravity, push, and pull forces



Physics forces acting on a Rigidbody: gravity pulling down, AddForce pushing sideways, and an impulse force for a jump/pull.

Procedure

1. Add a Rigidbody to the cube (use `Gravity` checked) so it is affected by gravity and falls onto the ground plane.
2. Add a second script that lets the player push or pull the cube with keys.

```
using UnityEngine;
```

```

[RequireComponent(typeof(Rigidbody))]
public class PhysicsForceDemo : MonoBehaviour
{
    public float pushForce = 8f;
    public float pullForce = 5f;
    private Rigidbody rb;

    void Start() => rb = GetComponent<Rigidbody>();

    void FixedUpdate()
    {
        if (Input.GetKey(KeyCode.P)) // push away

```

```

        rb.AddForce(transform.forward * pushForce, ForceMode.Force);

        if (Input.GetKey(KeyCode.O)) // pull toward player
            rb.AddForce(-transform.forward * pullForce, ForceMode.Force);

        if (Input.GetKeyDown(KeyCode.Space)) // upward impulse ("pop")
            rb.AddForce(Vector3.up * 6f, ForceMode.Impulse);
    }
}

```

Observation: Without any key held, the cube falls and settles under gravity alone. Holding P accelerates it forward continuously (a Force, mass-dependent and building speed over time); pressing Space gives it an instant upward Impulse (a sudden velocity change, as in a jump).

In my football project: the ball is a Rigidbody; ballplay.cs kicks it with `rb.AddForce(direction * kickForceplayer, ForceMode.Impulse)` and a tackle pushes it with `AddForceAtPosition`.

3(b) Audio triggered by movement/interaction events

Procedure

1. Add an AudioSource to the player character with a footstep clip assigned but **Play On Awake** unchecked.
2. Trigger playback from movement code, or precisely from an **Animation Event** on the walk cycle for footstep-to-foot-plant accuracy.

`using` UnityEngine;

```

public class FootstepAudio : MonoBehaviour
{
    public AudioSource audioSource;
    public AudioClip[] footstepClips;
    public float stepInterval = 0.4f;
    private float stepTimer;

    void Update()
    {
        bool isMoving = Mathf.Abs(Input.GetAxis("Horizontal")) > 0.1f
            || Mathf.Abs(Input.GetAxis("Vertical")) > 0.1f;

        if (isMoving)
        {
            stepTimer -= Time.deltaTime;
            if (stepTimer <= 0f)
            {
                PlayFootstep();
                stepTimer = stepInterval;
            }
        }
    }

    void PlayFootstep()
    {
        AudioClip clip = footstepClips[Random.Range(0, footstepClips.Length)];
        audioSource.PlayOneShot(clip);
    }
}

```

Observation: A footstep sound plays at a regular interval only while a movement key is held, giving audible feedback synchronized (approximately) with the character's stride.

[Insert screenshot]

Player Inspector: Audio Source + FootstepAudio script with clips assigned

3(c) Moving between scenes/levels by script

Procedure

1. Add both scenes (MainMenu, Level1) to File → Build Settings → Scenes In Build.
2. Attach the following script to a door/trigger volume or a UI button.

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class LevelTransition : MonoBehaviour
{
    public string nextSceneName = "Level1";

    private void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Player"))
        {
            SceneManager.LoadScene(nextSceneName);
        }
    }

    // Alternative: hooked up to a UI Button's onClick event
    public void LoadNextSceneButton() => SceneManager.LoadScene(nextSceneName);
}
```

Observation: Walking the tagged Player object into the trigger volume (or clicking the linked UI button) immediately swaps the active scene, demonstrating the level-transition mechanic.

[Insert screenshot]

Build Settings with both scenes listed + the trigger object Inspector

3(d) Scripting animations based on events/actions

Procedure

1. Build an Animator Controller with Idle, Walk, and Attack states and a **Speed** float plus an **Attack** trigger parameter.
2. Drive the parameters from the same input/event code already written.

using UnityEngine;

```
public class EventDrivenAnimation : MonoBehaviour
{
    private Animator animator;

    void Start() => animator = GetComponent<Animator>();

    void Update()
    {
        float speed = new Vector2(Input.GetAxis("Horizontal"), Input.GetAxis("Vertical")).magnitude;
        animator.SetFloat("Speed", speed);

        if (Input.GetMouseButtonDown(0)) // left-click = attack action
            animator.SetTrigger("Attack");
    }
}
```

Observation: The character blends from Idle to Walk automatically as Speed rises above the Animator's transition threshold, and interrupts into the Attack animation the instant the mouse button is clicked — confirming that gameplay events, not just continuous input, can drive the animation state machine.

In my football project: control.cs sets animator.SetBool("toRun", ...) from the input axes, and ballplay.cs sets "toKick" (Space with the ball), "toslide" (Space without the ball) and "spinSkill" (Left Shift) – the same event-to-animation pattern.

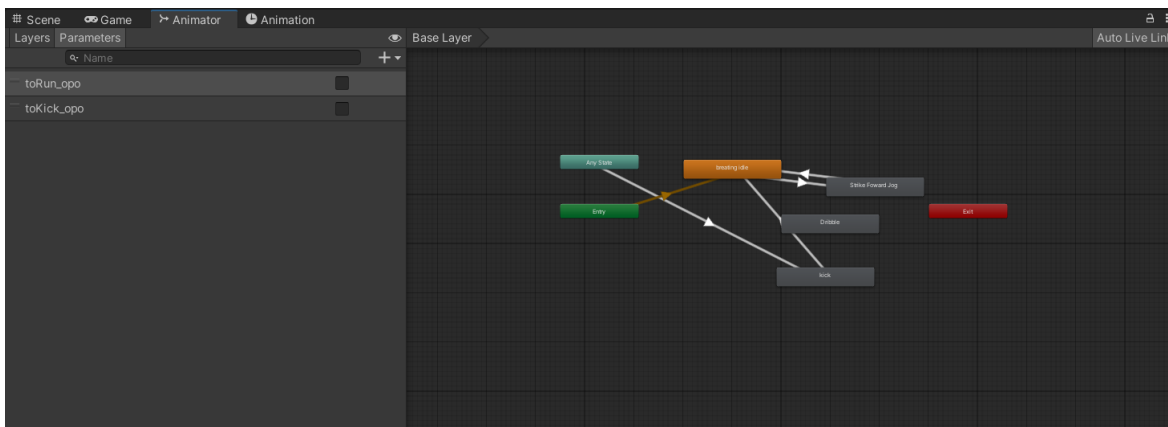
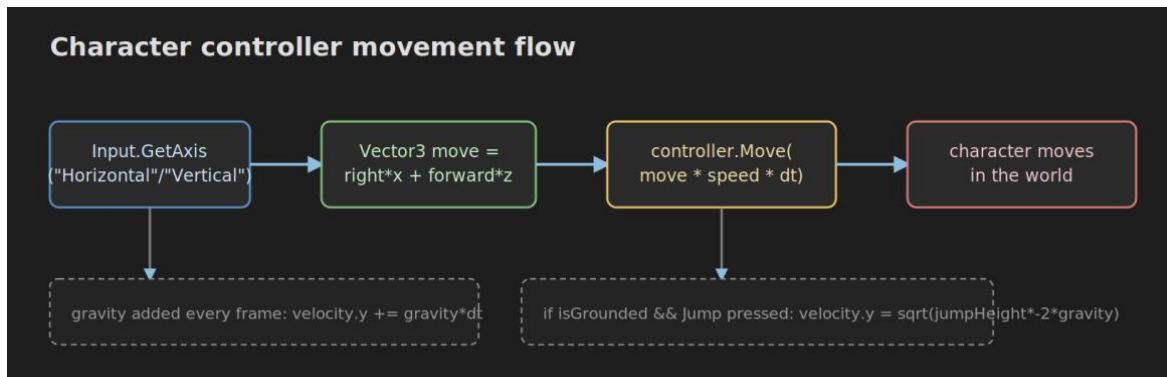


Fig. 3.1 – My project: Animator state machine of the opponent (Entry → breathing idle; Strike Forward Jog, Dribble, kick) with parameters toRun_opo and toKick_opo.

3(e) Full character: move left/right, jump, and crouch



Character controller movement flow from input to controller.Move, including gravity and jump branches.

```

using UnityEngine;

[RequireComponent(typeof(CharacterController))]
public class FullCharacterMovement : MonoBehaviour
{
    public float walkSpeed = 5f;
    public float crouchSpeed = 2.5f;
    public float jumpHeight = 1.4f;
    public float gravity = -9.81f;
    public float standingHeight = 2f;
    public float crouchingHeight = 1f;

    private CharacterController controller;
    private Vector3 velocity;
    private bool isCrouching;

    void Start() => controller = GetComponent<CharacterController>();

    void Update()
    {
        bool isGrounded = controller.isGrounded;
        if (isGrounded && velocity.y < 0) velocity.y = -2f;

        float x = Input.GetAxis("Horizontal");
        Vector3 move = transform.right * x;

        HandleCrouch();

        float currentSpeed = isCrouching ? crouchSpeed : walkSpeed;
        controller.Move(move * currentSpeed * Time.deltaTime);

        if (Input.GetButtonDown("Jump") && isGrounded && !isCrouching)
            velocity.y = Mathf.Sqrt(jumpHeight * -2f * gravity);

        velocity.y += gravity * Time.deltaTime;
        controller.Move(velocity * Time.deltaTime);
    }

    void HandleCrouch()
    {
        if (Input.GetKeyDown(KeyCode.LeftControl))
        {
            isCrouching = !isCrouching;
            controller.height = isCrouching ? crouchingHeight : standingHeight;
            controller.center = new Vector3(0, controller.height / 2f, 0);
        }
    }
}

```

```

    }
}

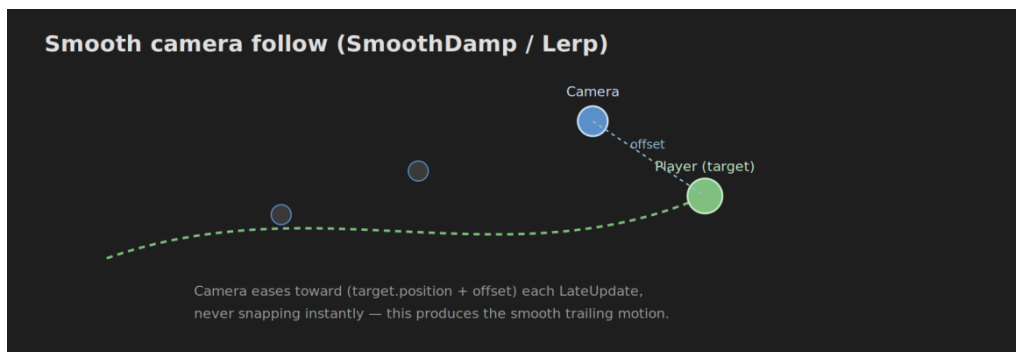
```

Observation: A/D (or Left/Right Arrow) strafes the character; Space jumps only while standing and grounded; toggling Left Ctrl shrinks the CharacterController's capsule height and halves movement speed, simulating a crouch that could allow passage under low obstacles. The `isGrounded` check prevents repeated jumping while in the air.

[Insert screenshot]

Player with Character Controller + FullCharacterMovement; Game view while jumping / crouching

3(f) Smooth camera following the player



Smooth camera follow: the camera eases toward the target's position plus an offset each `LateUpdate` instead of snapping instantly.

```

using UnityEngine;

public class SmoothCameraFollow : MonoBehaviour
{
    public Transform target;
    public Vector3 offset = new Vector3(0, 4f, -7f);
    public float smoothTime = 0.2f;
    private Vector3 velocity = Vector3.zero;

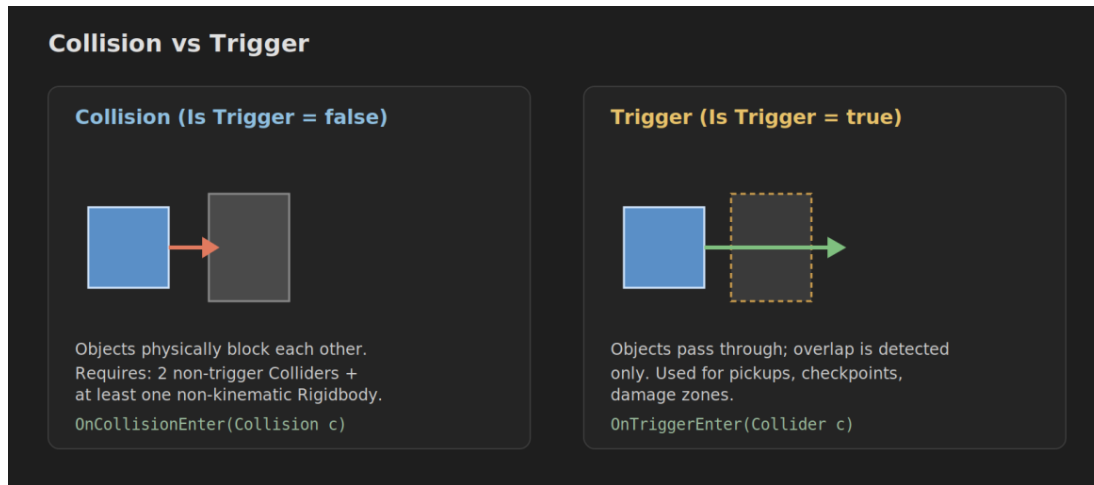
    void LateUpdate()
    {
        Vector3 desiredPosition = target.position + offset;
        transform.position = Vector3.SmoothDamp(transform.position, desiredPosition,
                                                ref velocity, smoothTime);
        transform.LookAt(target);
    }
}

```

Observation: The camera trails slightly behind the player's motion instead of rigidly locking to it, producing a natural "chase-cam" feel; increasing `smoothTime` makes the lag more pronounced, while a value near 0 approaches an instantly-snapping camera. Running this in `LateUpdate()` (rather than `Update()`) ensures the camera reads the player's already-updated position for that frame, avoiding a

one-frame jitter.

Experiment 4 — Platforms and Obstacles with Collision Detection



Collision versus Trigger comparison: solid blocking objects versus overlap-only detection zones.

Aim

To build a small set of platforms and obstacles and detect collisions between the player and each of them.

Theory

A **Collision** (non-trigger Collider + Rigidbody) physically blocks motion and fires `OnCollisionEnter/Stay/Exit`, while a **Trigger** (`Is Trigger` checked) allows overlap and fires `OnTriggerEnter/Stay/Exit` — the correct choice depends on whether the obstacle should be solid (a wall, a platform) or a detection-only zone (a hazard, a checkpoint).

Procedure

1. Build several Cube platforms at different heights using grid snapping, each with a `BoxCollider` (added automatically).
2. Add one obstacle with **Is Trigger** checked (e.g., a spike-trap zone that damages on overlap) and one without (a solid wall that blocks movement). Tag them `Hazard` and `Platform`.
3. Attach the script below to the player.

```
using UnityEngine;
```

```
public class PlatformCollisionHandler : MonoBehaviour
{
    public int health = 100;

    // Solid platforms / walls - physically block movement via Collider + Rigidbody
    private void OnCollisionEnter(Collision collision)
    {
        if (collision.gameObject.CompareTag("Platform"))
        {
            Debug.Log("Landed on platform: " + collision.gameObject.name);
        }
    }
}
```

```

    }

    // Trigger obstacle - hazard zone, does not block movement but damages on overlap
    private void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Hazard"))
        {
            health -= 20;
            Debug.Log("Hit hazard! Health: " + health);
        }
    }

    private void OnTriggerStay(Collider other)
    {
        if (other.CompareTag("Hazard"))
        {
            // continuous damage over time while standing in it
            health -= Mathf.RoundToInt(5 * Time.deltaTime);
        }
    }
}

```

Observation / Result: The player physically stands on and is blocked by solid platforms (Collision events fire, movement stops at the surface), while walking through the hazard trigger zone does not stop movement but reduces health (Trigger events fire) — demonstrating the practical difference between the two detection modes on the same scene.

Tip: `health -= Mathf.RoundToInt(5 * Time.deltaTime)` rounds to 0 every frame; use a float health variable (`healthF -= 5f * Time.deltaTime`) if continuous damage must be visible.

In my football project: the same mechanisms drive the game – `backtoplace.cs` on the goal nets uses `OnTriggerEnter` (ball enters the net – score +1, ball and players reset), `BoundaryController.cs` keeps objects inside the pitch, and `ballplay.OnCollisionEnter` detects the player colliding with the opponent (tackle drops the ball).

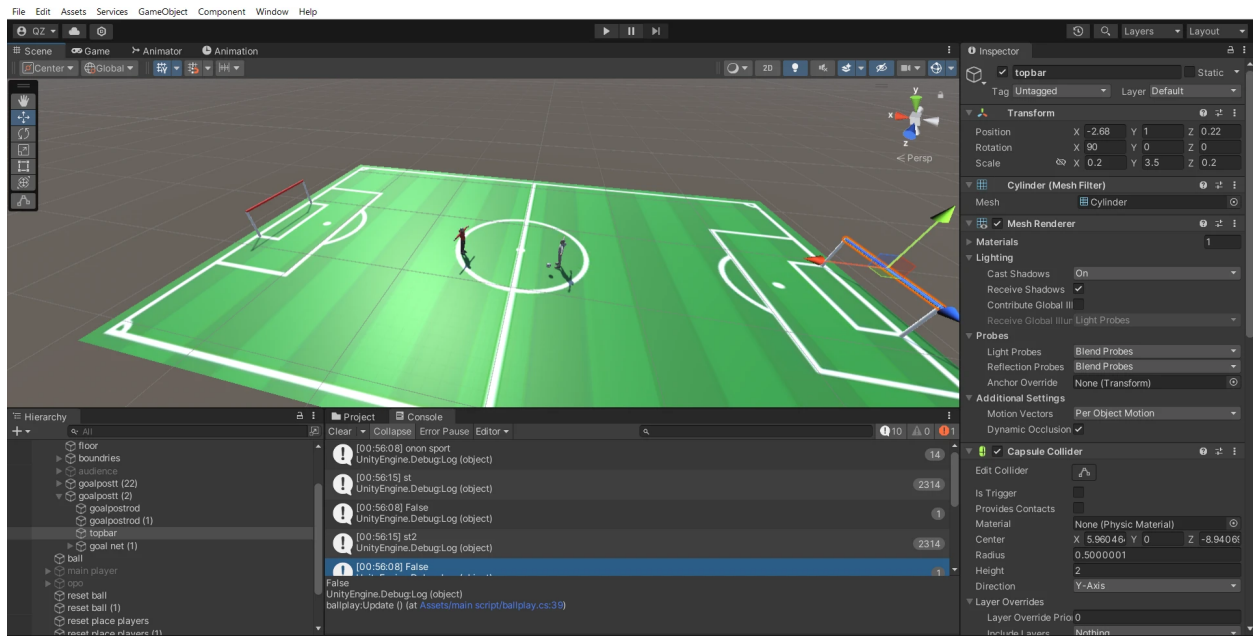


Fig. 4.2 – Goal post object (topbar) in the Hierarchy with its Capsule Collider (Is Trigger unticked = solid, blocks the ball); the Console shows the "onon sport" message printed by backtoplace.cs when the ball enters a goal net trigger (collapsed count 14).

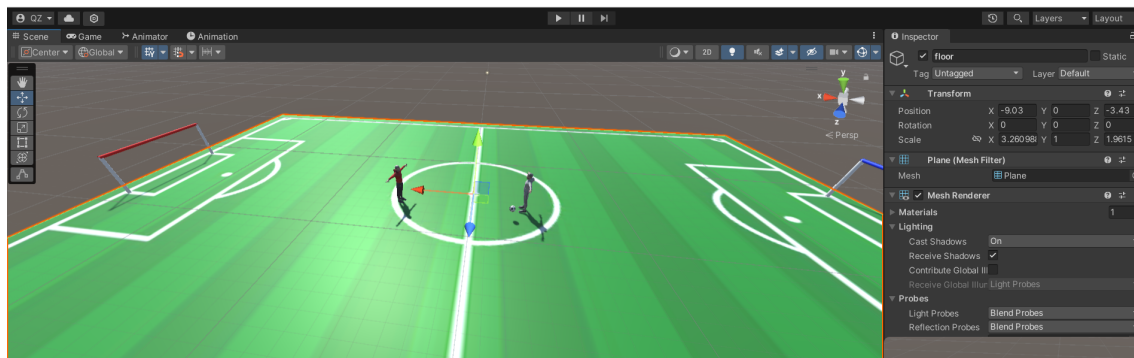


Fig. 4.1 – My project's level: pitch (platform), goal posts (trigger areas) and the start positions in the centre circle.

[Insert screenshot]

Platforms, hazard trigger and Console output "Hit hazard! Health: 80"

Experiment 5 — Animating the Player Character (Running, Jumping)

Aim

To apply and script running and jumping animations on an imported character, synchronized with actual movement state.

Procedure

1. Ensure the character's Animator Controller contains Idle, Run, and Jump states with a **Speed** Float parameter, a **Grounded** Bool and a **Jump** Trigger.
2. Attach the script below alongside the movement controller from Experiment 3(e).

```
using UnityEngine;

[RequireComponent(typeof(CharacterController))]
public class RunJumpAnimator : MonoBehaviour
{
    private Animator animator;
    private CharacterController controller;

    void Start()
    {
        animator = GetComponent<Animator>();
        controller = GetComponent<CharacterController>();
    }

    void Update()
    {
        float h = Input.GetAxis("Horizontal");
        float v = Input.GetAxis("Vertical");
        float speed = new Vector2(h, v).magnitude;

        // dampTime smooths Idle <-> Run blending
        animator.SetFloat("Speed", speed, 0.1f, Time.deltaTime);
        animator.SetBool("Grounded", controller.isGrounded);

        if (Input.GetButtonDown("Jump") && controller.isGrounded)
            animator.SetTrigger("Jump");
    }
}
```

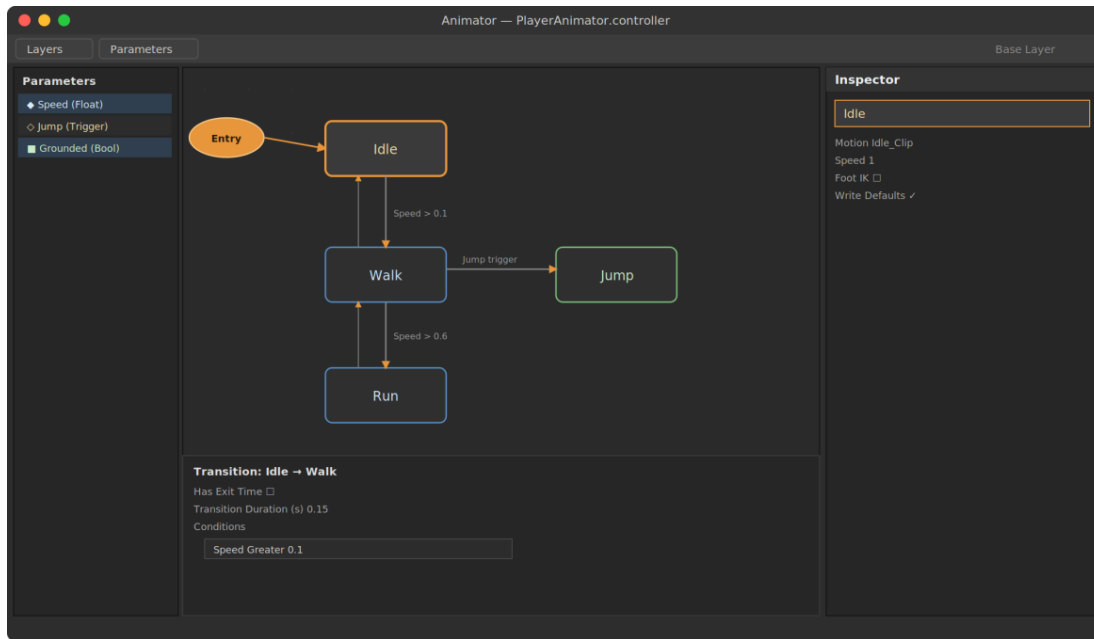
Observation / Result: As the player accelerates from standstill to full run speed, the Animator smoothly blends from the Idle pose into the Run cycle rather than popping abruptly; pressing Jump while grounded instantly plays the Jump animation, and the Grounded bool can be used by the Animator Controller's transitions to return automatically to Idle/Run upon landing.

In my football project: the running animation is switched by `animator.SetBool("toRun", true/false)` in `control.cs`; the clips (*Strike Foward Jog, Dribble, Soccer Spin*) are listed in Fig. 2.3. Jumping is not part of the football game, so the jump script above is demonstrated in a separate test scene.

[Insert screenshot]

Animator parameters (Speed, Grounded, Jump) and the character in Run / Jump state

Experiment 6 — Animation Transitions for Smoother Character Movement



Schematic of an Animator Controller state machine with Entry, Idle, Walk, Run, and Jump states connected by conditional transitions (illustration of the target setup).

Aim

To configure and tune transition settings between animation states so movement changes look fluid instead of abrupt.

Theory

Each transition arrow in the Animator window has its own **Conditions** (the parameter comparisons that trigger it), plus timing controls: **Exit Time** (what fraction of the current clip must play before the transition is allowed to start), **Transition Duration** (how long the cross-fade blend between the two clips takes), and **Has Exit Time** (whether the transition can only occur at a specific point in the clip, useful for attack/action animations that shouldn't be interrupted mid-swing).

Procedure

1. Select the Idle→Walk transition arrow in the Animator window.
2. Uncheck **Has Exit Time** for locomotion transitions (Idle↔Walk↔Run) so they can respond immediately to input changes rather than waiting for the current clip to finish.
3. Set **Transition Duration** to roughly 0.15–0.25 for a smooth cross-fade rather than an instant cut.
4. For the Attack transition, check **Has Exit Time** and set it close to 1.0, so the swing animation is allowed to complete before the character can transition back to Idle/Walk.
5. Convert the discrete Idle/Walk/Run states into a single 1D **Blend Tree** driven by the Speed float, which produces continuous interpolation between clips instead of discrete jumps — the smoothest possible locomotion result.

Script — verifying transition state from code

```
using UnityEngine;

public class TransitionMonitor : MonoBehaviour
{
    private Animator animator;

    void Start() => animator = GetComponent<Animator>();

    void Update()
    {
        AnimatorStateInfo state = animator.GetCurrentAnimatorStateInfo(0);
        bool inTransition = animator.IsInTransition(0);

        if (inTransition)
            Debug.Log("Currently blending between animation states...");
    }
}
```

Observation / Result: With Exit Time disabled and a short cross-fade duration on locomotion transitions, tapping movement keys quickly produces responsive, non-janky direction changes; the Blend Tree variant removes visible “seams” between Idle/Walk/Run entirely, since the character’s pose is a continuous interpolation of the neighboring clips rather than a hard state swap.

In my football project: the player and opponent controllers (*player_ani*, *opo_ani*) already use state-to-state transitions between *breathing idle*, the jog/run clip, *Dribble* and *kick* driven by Bool parameters (Fig. 3.1); Experiment 6 tunes their Exit Time and Transition Duration.

[Insert screenshot]

Transition Inspector: Has Exit Time unchecked, Duration 0.15, Condition Speed > 0.1

Experiment 7 — Player Movement with Unity’s Input System (Speed & Acceleration Tuning)

Aim

To implement basic player movement using Unity’s Input System package and experimentally compare different movement speed and acceleration values.

Theory

The New Input System (`com.unity.inputsystem`) is built around **Input Action Assets** — Action Maps containing named Actions bound to physical controls — replacing the older axis-name-based Input Manager with a more flexible, rebindable system.

Procedure

1. Install the Input System via Window → Package Manager → Unity Registry → Input System and accept the editor restart.
2. Create an Input Actions asset (Assets → Create → Input Actions), add a **Gameplay** Action Map with a **Move** Action (Value/Vector2, bound to WASD and the left analog stick).
3. Add a **PlayerInput** component to the character, referencing the created asset (Behavior = *Send Messages*).

```
using UnityEngine;
using UnityEngine.InputSystem;

public class TunableMovement : MonoBehaviour
{
    public float maxSpeed = 6f;
    public float acceleration = 12f;
    public float deceleration = 16f;

    private Vector2 moveInput;
    private Vector3 currentVelocity;

    public void OnMove(InputValue value) => moveInput = value.Get<Vector2>();

    void Update()
    {
        Vector3 targetDirection = new Vector3(moveInput.x, 0, moveInput.y).normalized;
        Vector3 targetVelocity = targetDirection * maxSpeed;

        float rate = (targetDirection.magnitude > 0.01f) ? acceleration : deceleration;
        currentVelocity = Vector3.MoveTowards(currentVelocity, targetVelocity,
                                             rate * Time.deltaTime);

        transform.Translate(currentVelocity * Time.deltaTime, Space.World);
    }
}
```

Experimenting with speed/acceleration values

Test	maxSpeed	acceleration	deceleration	Expected feel (confirm by testing)
A	3	30	30	Slow but instantly responsive — “arcade” tight
B	6	12	16	Moderate top speed with a short, natural ramp-up
C	10	4	4	Fast top speed but sluggish to start/stop — heavy, “icy” feel
D	6	100	100	Effectively instant velocity — snappy but can feel robotic

Observation / Result: Lower acceleration values relative to maxSpeed produce a perceptibly “heavier” character that takes visible time to reach full speed and to stop, useful for vehicles or large characters; high acceleration values approximate instant direction changes, better suited to fast-paced arcade controls. Tuning these two numbers independently (rather than only maxSpeed) is what separates “game feel” polish from a merely functional movement script.

[Insert screenshot]

Input Actions editor (Move: WASD + stick) and PlayerInput component

Experiment 8 — Measuring CPU Usage Across Game Scenarios

Aim

To measure and compare CPU usage in the Menu, Gameplay, and Cutscene scenarios of a game, and identify which scenario is the primary bottleneck.

Theory

The Unity Profiler (Window → Analysis → Profiler) breaks per-frame CPU time into categories — Scripts, Physics, Animation, Rendering, UI — letting a developer see exactly which subsystem consumes the largest share of the 16.6 ms budget needed for a steady 60 FPS.

Procedure

1. Open Window → Analysis → Profiler, enable the **CPU Usage** module, and optionally check **Deep Profile** for per-method breakdowns (Deep Profile adds overhead of its own, so only use it for diagnosis, not for measuring absolute frame times).
2. Enter Play Mode and navigate to the Menu scene; let it settle for a few seconds, then record the average FPS (Game view → **Stats**) and the CPU category breakdown.
3. Load into Gameplay and repeat the recording during typical play (movement, ball handling, active opponent).
4. Trigger a Cutscene / heavy-effects scene and repeat the recording.
5. Tabulate results and compare against the 16.6 ms (60 FPS) / 33.3 ms (30 FPS) budget lines.

Script — logging per-scenario CPU frame time to a CSV for later comparison

```
using UnityEngine;
using System.IO;

public class ScenarioCPULogger : MonoBehaviour
{
    public string scenarioName = "Gameplay";
    private float timer;
    private int frameCount;
    private float accumulatedTime;

    void Update()
    {
        frameCount++;
        accumulatedTime += Time.unscaledDeltaTime;
        timer += Time.unscaledDeltaTime;

        if (timer >= 1f) // log once per second
        {
```

```

float avgFrameMs = (accumulatedTime / frameCount) * 1000f;
string line = $"{System.DateTime.Now:HH:mm:ss},{scenarioName},"
              + $"{avgFrameMs:F2}ms,{1000f / avgFrameMs:F1}fps";
File.AppendAllText(Application.persistentDataPath + "/cpu_scenario_log.csv",
                  line + "\n");
timer = 0; frameCount = 0; accumulatedTime = 0;
    }
}
}

```

Results table (record your own measurements)

Scenario	Avg frame time (ms)	FPS	Dominant CPU category
Menu			
Gameplay			
Cutscene / Heavy effects			

In my football project: likely CPU hot-spots to check in the Profiler: (1) eight `Debug.Log` calls executed every frame in `ballplay.Update()` (confirmed in the Console, Fig. 8.1); (2) `rb.velocity` and rotation set from `Update()` in `control.cs` instead of `FixedUpdate()`; (3) `GameObject.FindGameObjectsWithTag` and repeated `GetComponent` calls in `backtoplace.cs` on every goal; (4) unused scripts/objects still in the scene.

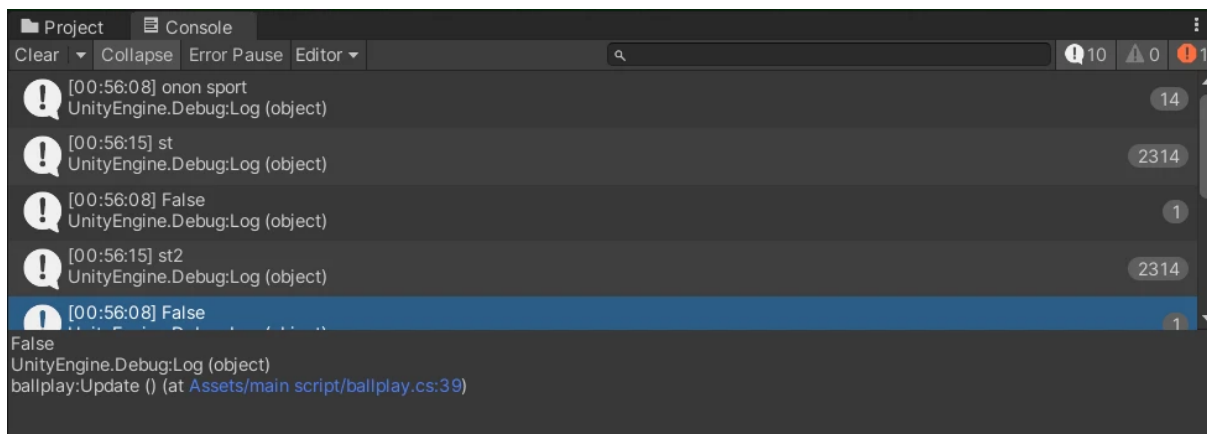


Fig. 8.1 – Console from my project during play: the per-frame debug lines from `ballplay.Update()` ("st", "st2") have each been logged 2314 times, plus `Debug.Log(False)` output – an avoidable CPU/GC cost found in the Console.

[Insert screenshot]

Profiler – CPU Usage module for Menu, Gameplay and Cutscene (one screenshot each)

Observation / Result: Write which scenario exceeds the 16.6 ms budget, which category dominates it (Scripts / Physics / Animation / Rendering) and what you would optimise first.

Experiment 9 — Monitoring GPU Usage and Analyzing Memory Consumption

Aim

To monitor GPU usage across gameplay segments and analyze overall memory consumption to ensure the game stays within a safe budget.

Theory

GPU cost is dominated by draw calls/batches (how many separate draw instructions the CPU must issue to the GPU), overdraw (pixels shaded more than once due to transparency/overlapping geometry), and shader complexity. Memory cost is dominated by texture memory, mesh memory, and garbage-collector allocations — frequent small allocations from `Update()`-loop code are a common hidden cause of periodic stutter even when average FPS looks acceptable.

Procedure — GPU monitoring

1. Open Window → Analysis → Profiler, enable the **GPU Usage** module (or use the **Frame Debugger**, Window → Analysis → Frame Debugger, to step through each draw call in a single frame).
2. Record **Draw Calls**, **Batches**, **SetPass Calls** and **Triangles/Vertices** during: (a) a normal gameplay view, (b) a high-object-density scene, (c) a scene with intensive lighting/effects.
3. Compare each segment against a defined GPU budget for the target platform (e.g., mobile: <150 draw calls).

Procedure — memory analysis

1. Enable the **Memory** module in the Profiler and take a Memory Snapshot (the Memory Profiler package, Window → Analysis → Memory Profiler, gives a more detailed breakdown) during each of the same segments.
2. Record **Total Allocated Memory**, **Texture Memory**, and **GC Alloc** per frame.

Script — runtime GPU/memory sampling utility

```
using UnityEngine;
using UnityEngine.Profiling;
using System.IO;

public class GPUMemoryMonitor : MonoBehaviour
{
    public string segmentName = "NormalGameplay";

    void Update()
    {
        if (Input.GetKeyDown(KeyCode.F9)) // manual sample trigger during a test run
        {
            long totalMemory = Profiler.GetTotalAllocatedMemoryLong();
            long graphicsMemory = Profiler.GetAllocatedMemoryForGraphicsDriver();

            string line = $"{segmentName},TotalMem={totalMemory / 1048576}MB,"
                + $"GraphicsMem={graphicsMemory / 1048576}MB,Time={Time.time:F1}s";

            File.AppendAllText(Application.persistentDataPath + "/gpu_mem_log.csv", line + "\n");
            Debug.Log("Sampled: " + line);
        }
    }
}
```

```
}
}
```

Results table (record your own measurements)

Segment	Draw calls	Batches	Triangles	Total memory	Texture memory	GC alloc / frame
Normal gameplay						
High-density scene						
Heavy lighting / effects						

[Insert screenshot]

Profiler GPU / Rendering / Memory modules, and Frame Debugger draw-call list

Observation / Result: Compare the segments: which has the most draw calls and GC allocation, and why (many separately rendered objects, un-pooled instantiation, real-time shadows). Link each finding to a fix, e.g. (1) GPU Instancing / static batching to reduce draw calls, (2) object pooling to remove per-frame allocations, (3) lower texture Max Size and fewer shadow-casting lights.

In my football project: candidates for optimisation: large pitch/audience textures (set Max Size, enable compression), real-time shadows from the Directional Light (reduce Shadow Distance), many identical objects (pooling / batching), and per-frame logging and Find... calls.

Overall Conclusion (Experiments 8–9)

Systematic per-scenario profiling — rather than a single averaged FPS number for the whole game — is essential to locate where a performance budget is actually being spent. Record measured values before and after each optimisation and keep the one that gives the largest gain for the least loss in quality, giving a clear, evidence-based priority order for optimisation work.

Appendix — Mapping to the Lab Assessment questions

Assessment question	Covered by
Q1 Asset integration, materials, shaders, lighting	Experiment 1, Experiment 2(c)
Q2 Terrain, character, animation, audio	Experiment 2, 2(a), 2(b), 3(b)
Q3 Player controller and camera system	Experiments 3(e), 3(f), 5, 6, 7
Q4 Level design, collision, scene management	Experiments 3(c), 4, 2(d)
Q5 Performance profiling and optimisation	Experiments 8 and 9